CampusCare: Digital Campus Healthcare Management System

Project Proposal for UCS503P
Submitted to: Ms. Anushka
Thapar Institute of Engineering and Technology

Lavdeep Singh, 1024030107 Harshveer Singh, 1024030109
Damanbir Singh, 1024030110 Kriti, 1024030111

August 19, 2026

1 Higher-order goal

The campus health center currently runs on manual processes, and this creates real friction for students and staff alike. Our goal is to reduce that time: shorter waits, easier access to medical history, and a consultation to dispensation process that doesn't depend on paper.

That's the broader aim. The concrete engineering goal, the one we can actually plan and build toward, is turning the health center's existing manual workflow into a system we can measure and deploy.

The system we're proposing ties together student identity, health records, doctor-specific queues, consultation notes, structured prescriptions, medicine dispensing, and administrative controls, all under one role-based platform.

We're deliberately staying out of clinical decision-making. Students and staff will still choose which doctor to see, and emergency cases will run through their own workflow instead of sitting in the regular queue.

2 Time to value

- We're building the smallest complete version of the healthcare workflow first: check-in, doctor queue, consultation, prescription, dispensing, and visit completion. That gives us something real to test early instead of a pile of half-finished features.
- A working end-to-end flow comes before trying to build every planned feature at once.
- Check-in will start out staff-assisted and manual, so we can validate the core workflow before we touch NFC or RFID integration at all.
- NFC/RFID support only gets added once we've actually tested whether the college ID card, the target Android device, the browser, and our deployment setup all work together.
- We'll work in short iterations with continuous testing, so problems in the core workflow surface early rather than after we've built more on top of them.
- CI/CD gets set up early so every change is automatically built, tested, and pushed to a staging environment.
- In the first iteration, success means a full healthcare visit can be processed correctly, not how many features got built.

3 Problem Statement

Right now, the campus health center leans heavily on manual processes for check-in, queue management, consultation records, prescriptions, and dispensing. That creates a handful of recurring problems:

- **Unclear queue status:** students often wait without knowing where they stand in line or how many people are ahead of them.
- **Limited access to medical history:** doctors don't always have quick access to a student's past visits, prescriptions, or relevant conditions, so consultations can start without that context.
- **Paper-based prescriptions:** handwriting is hard to read, papers get lost, and there's no structured digital record to fall back on during a future visit.
- **Weak dispensing records:** what a doctor prescribes and what a student actually receives aren't always tied together in one system.
- **Manual identity checks:** students without their ID card need staff to verify and check them in by hand.
- **Scattered information:** identity, health profile, visit history, prescriptions, queue state, and dispensing details often live across separate, disconnected processes.
- **Limited administrative oversight:** doctor availability, staff accounts, and card enrollment all need a central place to be managed.
- **Routine and emergency cases don't mix well:** the two need different handling, and forcing them through the same queue doesn't work.

Why this matters now: the health center deals with a steady stream of students but has limited medical and administrative staff. Making the queue more transparent, records easier to reach, and the workflow better coordinated can cut down on avoidable waiting and repeated paperwork, while making care more continuous for students.

4 Proposed Solution

4.1 Overview

We're building a **web-based Progressive Web App (PWA)** that digitizes the routine campus healthcare workflow, with clear roles for students, doctors, pharmacists/receptionists, and administrators.

The system will include:

- **Student portal:** students set up and maintain their profile, add relevant medical information, view their history and prescriptions, check in, pick an available doctor, and track their queue status.
- **Identity-assisted check-in:** we'll look into using the existing college ID card as an NFC/RFID check-in method, linking its identifier to the student's account. Staff-assisted manual check-in stays available whenever the card isn't an option.
- **Doctor-specific queues:** students join the queue of the doctor they've chosen and can see where they stand.

- **Doctor consultation workflow:** doctors call students in, pull up relevant history, log the consultation, write structured prescriptions, and redirect a student to another doctor's queue when needed.
- **Structured prescriptions:** prescriptions use dedicated fields for medication and dosage, plus a general instructions section, instead of relying purely on free-form text.
- **Pharmacist/receptionist workflow:** handles manual check-ins, verifies and logs what medicine was actually dispensed, and closes out completed visits.
- **Emergency recording:** emergency and trauma cases go through a separate workflow run by designated emergency staff, rather than joining the routine queue.
- **Administrative dashboard:** admins manage doctor and pharmacist/receptionist accounts, doctor availability, student-related admin info, and card enrollment.
- **Role-based access control:** what a user can see and do depends on their assigned role.
- **Audit logging:** important actions get recorded along with who did them and when, for accountability.

4.2 Core workflow

1. A student logs into CampusCare using their own profile.
2. At the health center, the student checks in with their college ID card if NFC/RFID support is live and working. If not, the pharmacist/receptionist checks them in manually.
3. The student picks an available doctor and joins that doctor's queue.
4. The student can check their queue status any time from their CampusCare account.
5. The doctor calls the next student when ready.
6. The doctor pulls up the student's health profile and past clinical records.
7. The doctor logs the consultation and writes a structured prescription.
8. If a student ends up with the wrong doctor, that doctor can redirect the visit to the right queue without creating a duplicate.
9. After the consultation, the visit moves into a dispensing state.
10. The pharmacist/receptionist verifies and logs the medicines actually dispensed.
11. The doctor's original prescription stays untouched as the clinical record. Any difference between what was prescribed and what was dispensed gets recorded separately, not overwritten.
12. Once dispensing is done, the visit closes and becomes part of the student's medical history.
13. Emergency and trauma cases skip the routine queue entirely and go through the separate emergency workflow.

4.3 Operational constraints for a campus setting

- The app needs to work well on the devices students and staff already have, so we're using responsive design and PWA features rather than assuming specific hardware.
- We're not trying to automate diagnosis or clinical decisions. The focus stays on workflow, access control, data integrity, and correctness.
- We'll rely on common cloud hosting and managed database services so the system doesn't need dedicated infrastructure to run.
- NFC/RFID check-in is built as a replaceable piece, so manual staff-assisted check-in still works if the hardware or browser support falls short.

5 Solution Approach

This is an engineering course project, so we're focused on workflow, system correctness, database design, security, authorization, queue management, and deployability, not on inventing new clinical algorithms.

5.1 Structured prescriptions and dispensing

Prescriptions will use structured fields rather than one open text box for everything.

Depending on how we finalize the model, each medication entry may include:

- Medication name
- Strength
- Dosage amount
- Dosage unit
- Form
- Route
- Frequency or schedule
- Duration
- Before/after food instructions where relevant
- As needed instructions where relevant
- Additional instructions

We'll settle on the exact structure after looking at established healthcare data models and figuring out which fields actually make sense for a campus health center.

The system keeps a clear line between what the doctor prescribed and what the pharmacist dispensed:

$$\text{Doctor's Prescription} \neq \text{Pharmacist's Dispensing Record}$$

That way the pharmacist can log what was actually handed out without ever quietly changing the doctor's original prescription.

We may draw on ideas from established healthcare interoperability standards like HL7 FHIR for the prescription structure, without trying to implement the full standard.

5.2 Web architecture

We're using a standard 3-tier architecture:

- **Frontend:** React, Vite, and Tailwind CSS, with separate interfaces for students, doctors, pharmacists/receptionists, and admins.
- **Backend API:** Python and FastAPI, handling authentication, authorization, student profiles, health records, check-in, queue management, consultations, prescriptions, dispensing, emergency records, RFID enrollment, and admin operations.
- **Database:** PostgreSQL, storing users, student profiles, health information, RFID credentials, visits, queue entries, consultations, prescriptions, dispensing records, emergency cases, and audit logs.

We're writing SQL by hand and managing schema changes through version-controlled migration files, rather than leaning on an ORM as the main database layer.

Business rules and authorization checks live on the backend, not just in the frontend.

5.3 Authentication and authorization

CampusCare gives students persistent accounts and gives staff authenticated accounts of their own.

There are four main roles:

- **Student**
- **Doctor**
- **Pharmacist/Receptionist**
- **Administrator**

Students can see their own medical information and visit history. Doctors, pharmacists/receptionists, and admins get access based on their responsibilities, following a permission matrix we'll finalize as we go.

We'll decide on the exact student authentication method, including whether to integrate with the college's Google identity system, once we've checked what's technically and institutionally feasible.

5.4 NFC/RFID identity integration

We're planning to use the existing college ID card as the physical identity mechanism for check-in.

The intended flow looks like this:

$$\text{College ID Card} \rightarrow \text{Card Identifier} \rightarrow \text{Student Account} \rightarrow \text{CampusCare Visit}$$

The card identifier itself isn't treated as the student's medical identity, it's just a pointer to their account.

Before we treat this as a confirmed dependency, we'll actually test whether the college card works with a browser-based Web NFC approach.

5.5 Queue management

Each doctor gets their own queue.

The queue system needs to support:

- Students joining a doctor's queue
- Viewing queue status
- Doctors calling the next student
- No-show handling
- Doctor redirection
- Queue completion
- Doctor availability

If a called student doesn't show up, we'll use a set buffer window. Once that window passes, the student gets moved to the end of the queue and the doctor moves on to the next person.

We'll decide on the exact no-show duration after watching how the actual health center workflow plays out.

5.6 CI/CD and faster delivery

We're using CI/CD to:

- Automatically build the frontend and validate the backend on every change.
- Run unit and integration tests.
- Handle linting and other automated checks.
- Produce repeatable deployment artifacts.
- Push successful builds to a staging environment.
- Catch integration issues between the four of us early, instead of finding them right before a demo.

6 Evaluation Criterion

We're measuring success with metrics tied directly to the core healthcare workflow.

6.1 Key performance indicator

Time to check-in to queue-entry: the time from when the student clicks check-in to when their queue entry is successfully created.

Events in the system will record timestamps:

$$\text{Check-in Started} \rightarrow \text{Identity Verified} \rightarrow \text{Doctor Selected} \rightarrow \text{Queue Entry Created}$$

We don't want to commit to a goal number upfront. Let's establish a baseline from the current manual process and see where we want to set our target.

- **Metric:** timestamps are created automatically by the system.
- **Baseline:** manual workflow as it exists today.
- **Attribution:** timestamps are from CampusCare directly, not estimates.

6.2 Secondary evaluation metrics

- **Queue transparency:** can students actually tell which doctor they're seeing, where they stand in the queue, and whether they've been called.
- **Waiting time:** time from joining the queue to being called.
- **Visit completion time:** time from check-in all the way through consultation, dispensing, and closing the visit.
- **Record accessibility:** how long it takes, or how many steps it takes, for a doctor to pull up a student's relevant past records.
- **Prescription completeness:** what percentage of prescriptions include all the required structured fields.
- **Dispensing accuracy:** what percentage of visits correctly record prescribed versus dispensed medicine, including any documented differences.
- **Reliability:** how available and stable the system is during the pilot period.
- **Authorization correctness:** how well the system blocks actions outside a user's role.

6.3 Pilot validation plan

- Use synthetic student and medical data for development, testing, and demos.
- Only use real student medical data if the institution explicitly signs off and our deployment meets the required privacy and security standards.
- Bring in a representative mix of student, doctor, pharmacist/receptionist, and admin test users.
- Walk through real workflows: check-in, doctor selection, queue monitoring, consultation, prescription creation, dispensing, visit completion, doctor redirection, no-show handling, emergency recording, and admin tasks.
- Compare the system's metrics against what we observe from the existing manual process.

7 Scalability

We want this to scale both in theory and in practice.

1. **Scalable in theory:** the system should handle growth in students, visits, doctors, records, and concurrent sessions by:
    - keeping the frontend and backend separate,
    - paginating large result sets,
    - indexing common database queries,
    - keeping the relational data normalized,
    - using transactions for queue and visit state changes,
    - keeping the API stateless wherever practical.
2. **Deployable in practice:** the system should run on common web hosting and managed database infrastructure:

- a managed PostgreSQL database,
- a platform-hosted frontend and backend,
- environment-based configuration,
- a CI/CD pipeline for staging and production.

8 Engine availability heuristic

We already have an existing battle-tested engine that we can use to build CampusCare as a web application:

1. **React + Vite:** for frontend development.
2. **Tailwind CSS:** for composable responsive interfaces.
3. **FastAPI:** for our REST API and server logic.
4. **PostgreSQL:** for our relational database.
5. **Raw SQL alongside migration tooling:** for explicit schema development and versioning.
6. **Web NFC:** for browser-based NFC interactions. This is the approach we'll try to validate with the physical college ID card and Android phone.
7. **Authentication infrastructure:** to handle authentication so students and staff have persistent accounts.
8. **Testing tooling:** for unit tests and integration tests.
9. **CI/CD tooling:** to automate checks and deploy our services.
10. **Managed cloud hosting:** to deploy our frontend and API.
11. **Managed PostgreSQL hosting:** to put a cloud-hosted database into staging, testing, or production.

Building off of existing tools means we can focus on workflow, correctness, security, database design, and measurement.

9 Project scope and deliverables

9.1 Initial deliverable

The first iteration covers the smallest complete healthcare workflow:

- Student account and authentication
- Student profile
- Staff-assisted manual check-in
- Doctor selection
- One doctor's queue
- Queue position and status
- Doctor authentication

- Doctor calls the next student
- Doctor views relevant student medical history
- Doctor records the consultation
- Doctor creates a structured prescription
- Pharmacist/receptionist views the prescription
- Pharmacist/receptionist records medicine dispensing
- Visit completion
- Completed visit shows up in student history
- PostgreSQL persistence
- Backend REST API
- Basic role-based authorization
- CI build and backend tests
- Deployment to a staging environment

The point here is proving a complete healthcare visit can move through the system successfully, not cramming in as many screens or optional features as possible.

9.2 Subsequent deliverables

- A second doctor with an independent queue.
- Doctor availability management.
- Doctor redirection between queues.
- No-show handling and queue reordering.
- Full student medical history and prescription history.
- A better clinical history interface for doctors.
- A more complete set of role-based permission rules.
- NFC/RFID check-in.
- RFID enrollment and revocation.
- Card-to-student identifier mapping.
- Emergency and trauma recording.
- Administrative dashboard.
- Improvements to the pharmacist dispensing workflow.
- Audit logging.
- Polling-based live queue updates.
- WebSocket-based queue updates if time allows.
- More testing and security hardening.

9.3 Stretch deliverables

Following the initial launch of the core system, additional features may be considered, such as:

- **Medication reminders:** reminders generated based on an individual's prescribed medications and their respective schedules.
- **Syndrome surveillance:** a system that uses aggregated and de-identified visits information to detect and report unusual increases in a particular illness.

9.4 Explicitly out of scope

We won't be building **medicine inventory management** this semester.

It's a natural next step since it would tie dispensing records to actual stock levels, but we're leaving it out on purpose to keep the core system achievable within the semester.

10 Risks and mitigations

- **NFC/Web NFC compatibility:** the college card might not expose the identifier we need through the browser. We'll run an early feasibility test with the actual card, device, browser, and HTTPS deployment before treating NFC as a hard dependency.
- **Medical data privacy:** medical records are sensitive. We'll stick to synthetic data for development and demos unless the institution explicitly signs off on real data and we have the right privacy and security controls in place.
- **Unauthorized access to medical records:** different roles need different levels of access. We'll enforce role-based authorization on the backend, with a clear permission matrix, before exposing any medical records.
- **Prescription tampering:** the pharmacist should never be able to quietly change a doctor's clinical prescription. That's why prescriptions and dispensing records are stored separately.
- **Queue inconsistency:** concurrent requests could cause race conditions in the queue. We'll use PostgreSQL transactions and proper concurrency controls for queue state changes.
- **Scope creep:** NFC, WebSockets, reminders, analytics, emergency handling, admin tools, and structured prescriptions could easily blow past the semester timeline together. Core functionality comes first, stretch features only after.
- **Student identity mismatch:** we shouldn't rely purely on unrestricted manual input for student identity. We'll prefer institutional authentication or verified student information wherever it's available.
- **No-show handling:** a student who doesn't show up after being called can block the whole queue. A defined buffer period fixes this, moving them to the back of the line once it expires.
- **Doctor availability changes:** changing a doctor's availability while students are already queued could leave things in an inconsistent state. We'll define clear rules for existing queues before building availability management.
- **Deployment issues:** cloud deployment or database problems could disrupt a demo. We're setting up staging deployment early and keeping our deployment config version-controlled through CI/CD.

11 Summary

CampusCare is a deployable web platform meant to make routine healthcare on campus more organized and transparent.

It ties together student identity, optional NFC/RFID-assisted check-in, doctor-specific queues, digital medical history, structured prescriptions, pharmacist dispensing, and admin controls, all within a role-based system.

The focus here is **engineering workflow and system correctness, not clinical automation**. Routine consultations follow a controlled path from check-in to queue, consultation, prescription, dispensing, and completion, while emergency and trauma cases run through their own separate workflow.

We're starting with a walking skeleton: manual check-in and one complete doctor workflow first, then gradually adding a second doctor, queue management features, NFC/RFID integration, emergency recording, admin functionality, and better real-time updates.

On the technical side, we're using React, Vite, and Tailwind on the frontend, FastAPI for the backend, and PostgreSQL with raw SQL and version-controlled migrations. CI/CD, automated testing, role-based authorization, transaction-safe queue management, and audit logging support the whole build as we go.

We'll evaluate the workflow on measurable terms: check-in-to-queue-entry time, waiting time, queue transparency, visit completion, prescription completeness, dispensing accuracy, system reliability, and authorization correctness.

Medicine inventory management is intentionally out of scope this semester, while medication reminders and anonymized disease-surge indicators remain optional stretch goals.

Overall, the plan is to deliver a **complete, secure, measurable healthcare workflow first**, and only add more once the core system is solid.